# Histograms - 1 : Find, Plot, Analyze !!!

## Goal

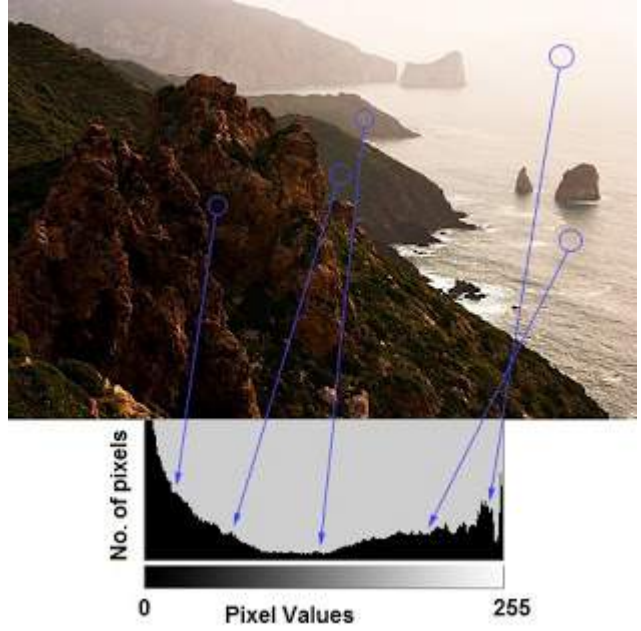
Learn to

- Find histograms, using both OpenCV and Numpy functions
- Plot histograms, using OpenCV and Matplotlib functions
- You will see these functions : **cv.calcHist()**, **np.histogram()** etc.

## Theory

So what is histogram ? You can consider histogram as a graph or plot, which gives you an overall idea about the intensity distribution of an image. It is a plot with pixel values (ranging from 0 to 255, not always) in X-axis and corresponding number of pixels in the image on Y-axis.

It is just another way of understanding the image. By looking at the histogram of an image, you get intuition about contrast, brightness, intensity distribution etc of that image. Almost all image processing tools today, provides features on histogram. Below is an image from Cambridge in Color website, and I recommend you to visit the site for more details.




**image**

You can see the image and its histogram. (Remember, this histogram is drawn for grayscale image, not color image). Left region of histogram shows the amount of darker pixels in image and right region shows the amount of brighter pixels. From the histogram, you can see dark region is more than brighter region, and amount of midtones (pixel values in mid-range, say around 127) are very less.

# Find Histogram

Now we have an idea on what is histogram, we can look into how to find this. Both OpenCV and Numpy come with in-built function for this. Before using those functions, we need to understand some terminologies related with histograms.

**BINS** :The above histogram shows the number of pixels for every pixel value, ie from 0 to 255. ie you need 256 values to show the above histogram. But consider, what if you need not find the number of pixels for all pixel values separately, but number of pixels in a interval of pixel values? say for example, you need to find the number of pixels lying between 0 to 15, then 16 to 31, ..., 240 to 255. You will need only 16 values to represent the histogram. And that is what is shown in example given in **OpenCV Tutorials on histograms**.

So what you do is simply split the whole histogram to 16 sub-parts and value of each sub-part is the sum of all pixel count in it. This each sub-part is called "BIN". In first case, number of bins were 256 (one for each pixel) while in second case, it is only 16. BINS is represented by the term **histSize** in OpenCV docs.

**DIMS** : It is the number of parameters for which we collect the data. In this case, we collect data regarding only one thing, intensity value. So here it is 1.

**RANGE** : It is the range of intensity values you want to measure. Normally, it is [0,256], ie all intensity values.

## 1. Histogram Calculation in OpenCV

So now we use **cv.calcHist()** function to find the histogram. Let's familiarize with the function and its parameters :

*cv.calcHist(images, channels, mask, histSize, ranges[, hist[, accumulate]])*

1. images : it is the source image of type uint8 or float32. it should be given in square brackets, ie, "[img]".
2. channels : it is also given in square brackets. It is the index of channel for which we calculate histogram. For example, if input is grayscale image, its value is [0]. For color image, you can pass [0], [1] or [2] to calculate histogram of blue, green or red channel respectively.
3. mask : mask image. To find histogram of full image, it is given as "None". But if you want to find histogram of particular region of image, you have to create a mask image for that and give it as mask. (I will show an example later.)
4. histSize : this represents our BIN count. Need to be given in square brackets. For full scale, we pass [256].
5. ranges : this is our RANGE. Normally, it is [0,256].

So let's start with a sample image. Simply load an image in grayscale mode and find its full histogram.

```
img = cv.imread('home.jpg', cv.IMREAD_GRAYSCALE)
assert img is not None, "file could not be read, check with os.path.exists()"
hist = cv.calcHist([img],[0],None,[256],[0,256])
```

hist is a 256x1 array, each value corresponds to number of pixels in that image with its corresponding pixel value.

## 2. Histogram Calculation in Numpy

Numpy also provides you a function, **np.histogram()**. So instead of calcHist() function, you can try below line :

```
hist,bins = np.histogram(img.ravel(),256,[0,256])
```

hist is same as we calculated before. But bins will have 257 elements, because Numpy calculates bins as 0-0.99, 1-1.99, 2-2.99 etc. So final range would be 255-255.99. To represent that, they also add 256 at end of bins. But we don't need that 256. Upto 255 is sufficient.

**Note**

Numpy has another function, **np.bincount()** which is much faster than (around 10X) np.histogram(). So for one-dimensional histograms, you can better try that. Don't forget to set minlength = 256 in np.bincount. For example, hist = np.bincount(img.ravel(),minlength=256)

OpenCV function is faster than (around 40X) than np.histogram(). So stick with OpenCV function.

Now we should plot histograms, but how?

# Plotting Histograms

There are two ways for this,

1. Short Way : use Matplotlib plotting functions
2. Long Way : use OpenCV drawing functions

## 1. Using Matplotlib

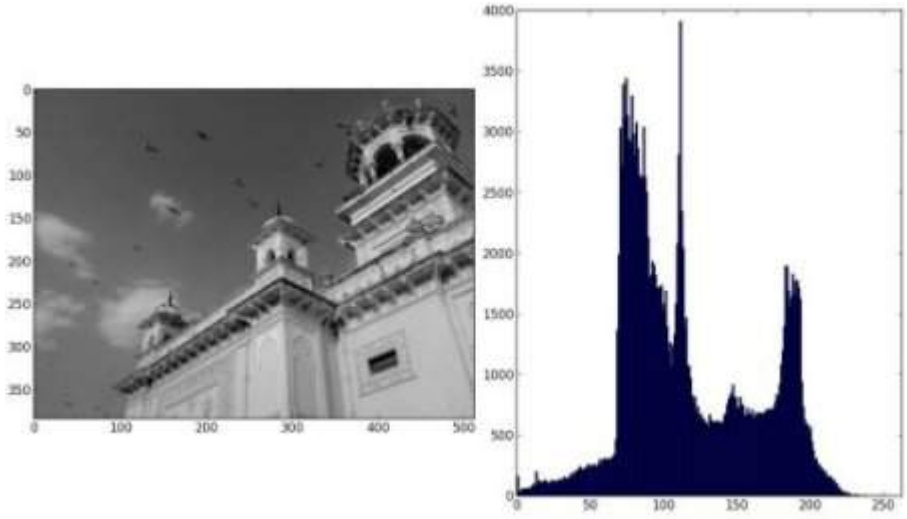
Matplotlib comes with a histogram plotting function : matplotlib.pyplot.hist()

It directly finds the histogram and plot it. You need not use calcHist() or np.histogram() function to find the histogram. See the code below:

```
import numpy as np
import cv2 as cv
from matplotlib import pyplot as plt

img = cv.imread('home.jpg', cv.IMREAD_GRAYSCALE)
assert img is not None, "file could not be read, check with os.path.exists()"
plt.hist(img.ravel(),256,[0,256]); plt.show()
```

You will get a plot as below :



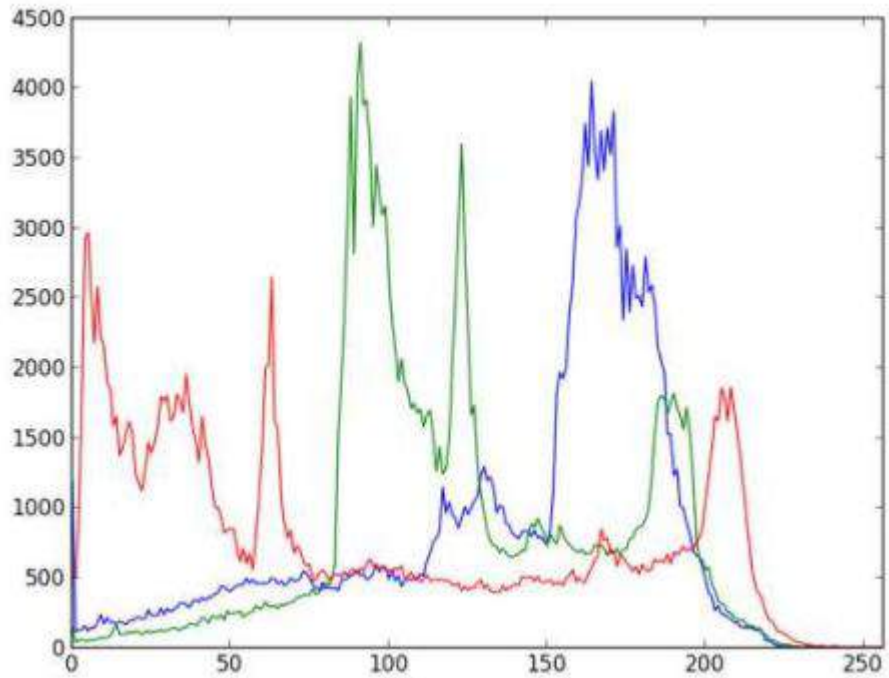
**image**

Or you can use normal plot of matplotlib, which would be good for BGR plot. For that, you need to find the histogram data first. Try below code:

```
import numpy as np
import cv2 as cv
from matplotlib import pyplot as plt

img = cv.imread('home.jpg')
assert img is not None, "file could not be read, check with os.path.exists()"
color = ('b','g','r')
for i,col in enumerate(color):
    histr = cv.calcHist([img],[i],None,[256],[0,256])
    plt.plot(histr,color = col)
    plt.xlim([0,256])
plt.show()
```

Result:




**image**

You can deduct from the above graph that, blue has some high value areas in the image (obviously it should be due to the sky)

## 2. Using OpenCV

Well, here you adjust the values of histograms along with its bin values to look like x,y coordinates so that you can draw it using **cv.line()** or cv.polyline() function to generate same image as above. This is already available with OpenCV-Python2 official samples. Check the code at samples/python/hist.py.

# Application of Mask

We used **cv.calcHist()** to find the histogram of the full image. What if you want to find histograms of some regions of an image? Just create a mask image with white color on the region you want to find histogram and black otherwise. Then pass this as the mask.

```
img = cv.imread('home.jpg', cv.IMREAD_GRAYSCALE)
assert img is not None, "file could not be read, check with os.path.exists()"

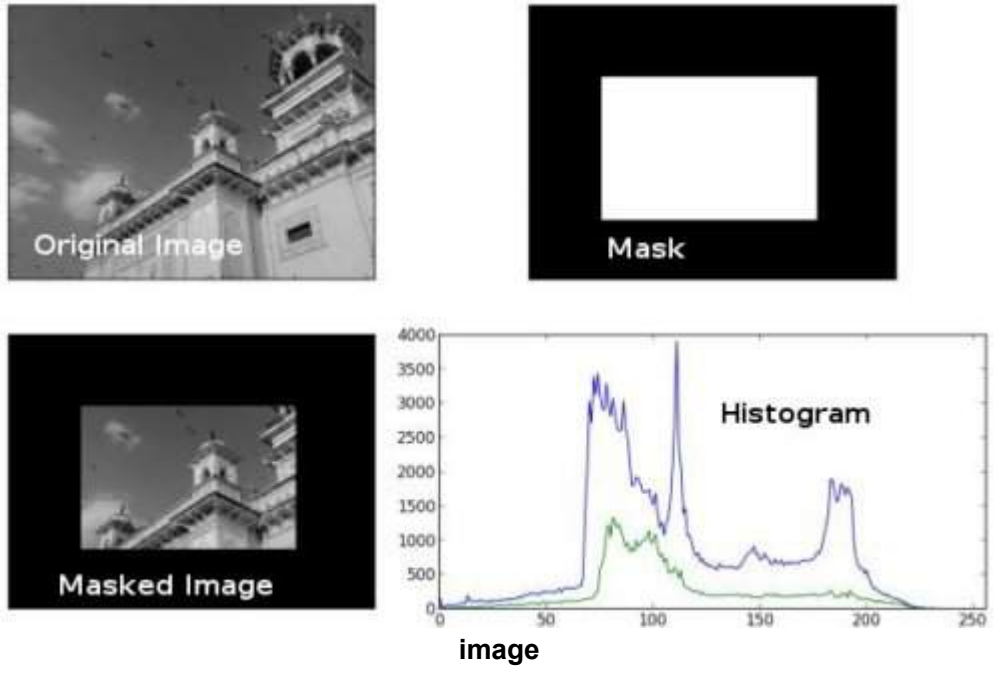
# create a mask
mask = np.zeros(img.shape[:2], np.uint8)
mask[100:300, 100:400] = 255
masked_img = cv.bitwise_and(img,img,mask = mask)

# Calculate histogram with mask and without mask
# Check third argument for mask
hist_full = cv.calcHist([img],[0],None,[256],[0,256])
hist_mask = cv.calcHist([img],[0],mask,[256],[0,256])

plt.subplot(221), plt.imshow(img, 'gray')
plt.subplot(222), plt.imshow(mask,'gray')
plt.subplot(223), plt.imshow(masked_img, 'gray')
plt.subplot(224), plt.plot(hist_full), plt.plot(hist_mask)
plt.xlim([0,256])

plt.show()
```

See the result. In the histogram plot, blue line shows histogram of full image while green line shows histogram of masked region.




**image**

# Additional Resources

1. Cambridge in Color website

# Exercises

ENHANCED BY Google



# Histograms - 2: Histogram Equalization

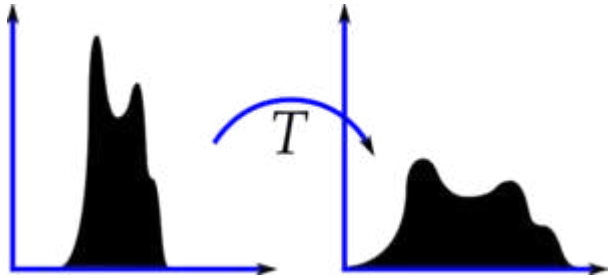
## Goal

In this section,

- We will learn the concepts of histogram equalization and use it to improve the contrast of our images.

## Theory

Consider an image whose pixel values are confined to some specific range of values only. For eg, brighter image will have all pixels confined to high values. But a good image will have pixels from all regions of the image. So you need to stretch this histogram to either ends (as given in below image, from wikipedia) and that is what Histogram Equalization does (in simple words). This normally improves the contrast of the image.




**image**

I would recommend you to read the wikipedia page on Histogram Equalization for more details about it. It has a very good explanation with worked out examples, so that you would understand almost everything after reading that. Instead, here we will see its Numpy implementation. After that, we will see OpenCV function.

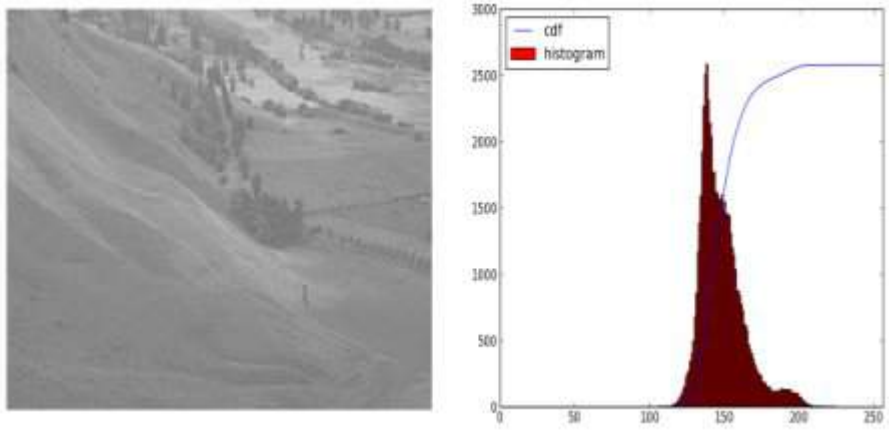
```
import numpy as np
import cv2 as cv
from matplotlib import pyplot as plt

img = cv.imread('wiki.jpg', cv.IMREAD_GRAYSCALE)
assert img is not None, "file could not be read, check with os.path.exists()"

hist,bins = np.histogram(img.flatten(),256,[0,256])

cdf = hist.cumsum()
cdf_normalized = cdf * float(hist.max()) / cdf.max()

plt.plot(cdf_normalized, color = 'b')
plt.hist(img.flatten(),256,[0,256], color = 'r')
plt.xlim([0,256])
plt.legend(('cdf','histogram'), loc = 'upper left')
plt.show()
```




You can see histogram lies in brighter region. We need the full spectrum. For that, we need a transformation function which maps the input pixels in brighter region to output pixels in full region. That is what histogram equalization does.

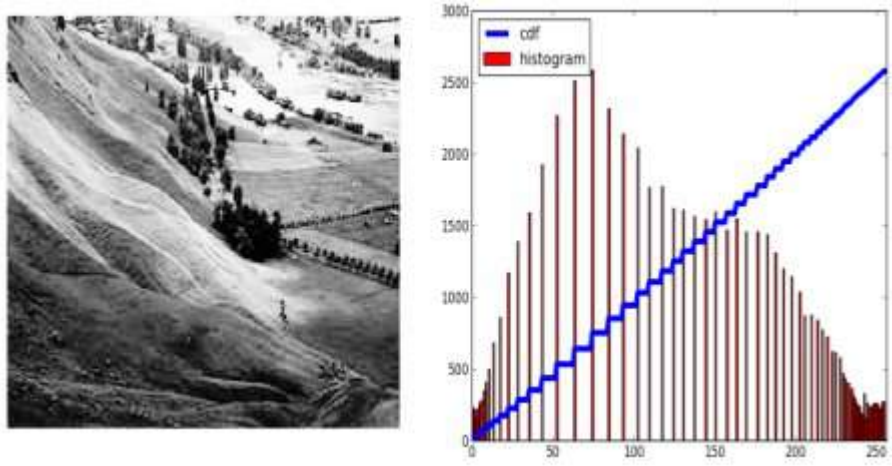
Now we find the minimum histogram value (excluding 0) and apply the histogram equalization equation as given in wiki page. But I have used here, the masked array concept array from Numpy. For masked array, all operations are performed on non-masked elements. You can read more about it from Numpy docs on masked arrays.

```
cdf_m = np.ma.masked_equal(cdf,0)
cdf_m = (cdf_m - cdf_m.min())*255/(cdf_m.max()-cdf_m.min())
cdf = np.ma.filled(cdf_m,0).astype('uint8')
```

Now we have the look-up table that gives us the information on what is the output pixel value for every input pixel value. So we just apply the transform.

```
img2 = cdf[img]
```

Now we calculate its histogram and cdf as before ( you do it) and result looks like below :




**image**

Another important feature is that, even if the image was a darker image (instead of a brighter one we used), after equalization we will get almost the same image as we got. As a result, this is used as a "reference tool" to make all images with same lighting conditions. This is useful in many cases. For example, in face recognition, before training the face data, the images of faces are histogram equalized to make them all with same lighting conditions.

# Histograms Equalization in OpenCV

OpenCV has a function to do this, **cv.equalizeHist()**. Its input is just grayscale image and output is our histogram equalized image.

Below is a simple code snippet showing its usage for same image we used :

```
img = cv.imread('wiki.jpg', cv.IMREAD_GRAYSCALE)
assert img is not None, "file could not be read, check with os.path.exists()"
equ = cv.equalizeHist(img)
res = np.hstack((img,equ)) #stacking images side-by-side
cv.imwrite('res.png',res)
```



So now you can take different images with different light conditions, equalize it and check the results.

Histogram equalization is good when histogram of the image is confined to a particular region. It won't work good in places where there is large intensity variations where histogram covers a large region, ie both bright and dark pixels are present. Please check the SOF links in Additional Resources.

# CLAHE (Contrast Limited Adaptive Histogram Equalization)

The first histogram equalization we just saw, considers the global contrast of the image. In many cases, it is not a good idea. For example, below image shows an input image and its result after global histogram equalization.

Original Image



After Global
Histogram Equalization



**image**

It is true that the background contrast has improved after histogram equalization. But compare the face of statue in both images. We lost most of the information there due to over-brightness. It is because its histogram is not confined to a particular region as we saw in previous cases (Try to plot histogram of input image, you will get more intuition).

So to solve this problem, **adaptive histogram equalization** is used. In this, image is divided into small blocks called "tiles" (tileSize is 8x8 by default in OpenCV). Then each of these blocks are histogram equalized as usual. So in a small area, histogram would confine to a small region (unless there is noise). If noise is there, it will be amplified. To avoid this, **contrast limiting** is applied. If any histogram bin is above the specified contrast limit (by default 40 in OpenCV), those pixels are clipped and distributed uniformly to other bins before applying histogram equalization. After equalization, to remove artifacts in tile borders, bilinear interpolation is applied.

Below code snippet shows how to apply CLAHE in OpenCV:

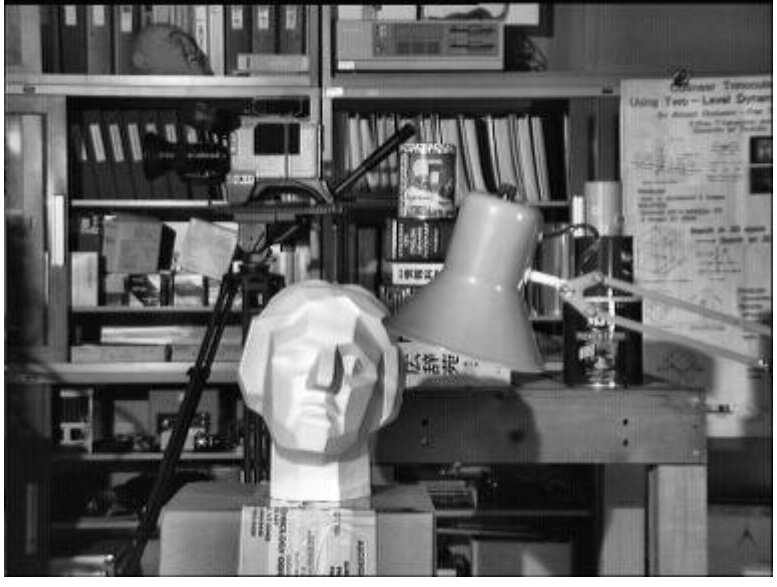
```
import numpy as np
import cv2 as cv

img = cv.imread('tsukuba_l.png', cv.IMREAD_GRAYSCALE)
assert img is not None, "file could not be read, check with os.path.exists()"

# create a CLAHE object (Arguments are optional).
clahe = cv.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
cl1 = clahe.apply(img)

cv.imwrite('clahe_2.jpg',cl1)
```

See the result below and compare it with results above, especially the statue region:



**image**

# Additional Resources

1. Wikipedia page on Histogram Equalization
2. Masked Arrays in Numpy

Also check these SOF questions regarding contrast adjustment:

1. How can I adjust contrast in OpenCV in C?
2. How do I equalize contrast & brightness of images using opencv?

# Exercises

ENHANCED BY Google



# Histograms - 3 : 2D Histograms

## Goal

In this chapter, we will learn to find and plot 2D histograms. It will be helpful in coming chapters.

## Introduction

In the first article, we calculated and plotted one-dimensional histogram. It is called one-dimensional because we are taking only one feature into our consideration, ie grayscale intensity value of the pixel. But in two-dimensional histograms, you consider two features. Normally it is used for finding color histograms where two features are Hue & Saturation values of every pixel.

There is a python sample (samples/python/color_histogram.py) already for finding color histograms. We will try to understand how to create such a color histogram, and it will be useful in understanding further topics like Histogram Back-Projection.

## 2D Histogram in OpenCV

It is quite simple and calculated using the same function, **cv.calcHist()**. For color histograms, we need to convert the image from BGR to HSV. (Remember, for 1D histogram, we converted from BGR to Grayscale). For 2D histograms, its parameters will be modified as follows:

- **channels = [0,1]** *because we need to process both H and S plane.*
- **bins = [180,256]** *180 for H plane and 256 for S plane.*
- **range = [0,180,0,256]** *Hue value lies between 0 and 180 & Saturation lies between 0 and 256.*

Now check the code below:

```
import numpy as np
import cv2 as cv

img = cv.imread('home.jpg')
assert img is not None, "file could not be read, check with os.path.exists()"
hsv = cv.cvtColor(img,cv.COLOR_BGR2HSV)

hist = cv.calcHist([hsv], [0, 1], None, [180, 256], [0, 180, 0, 256])
```

That's it.

# 2D Histogram in Numpy

Numpy also provides a specific function for this : **np.histogram2d()**. (Remember, for 1D histogram we used **np.histogram()** ).

```
import numpy as np
import cv2 as cv
from matplotlib import pyplot as plt

img = cv.imread('home.jpg')
assert img is not None, "file could not be read, check with os.path.exists()"
hsv = cv.cvtColor(img,cv.COLOR_BGR2HSV)

hist, xbins, ybins = np.histogram2d(h.ravel(),s.ravel(),[180,256],[[0,180],[0,256]])
```

First argument is H plane, second one is the S plane, third is number of bins for each and fourth is their range.

Now we can check how to plot this color histogram.

# Plotting 2D Histograms

## Method - 1 : Using cv.imshow()

The result we get is a two dimensional array of size 180x256. So we can show them as we do normally, using **cv.imshow()** function. It will be a grayscale image and it won't give much idea what colors are there, unless you know the Hue values of different colors.

## Method - 2 : Using Matplotlib

We can use **matplotlib.pyplot.imshow()** function to plot 2D histogram with different color maps. It gives us a much better idea about the different pixel density. But this also, doesn't gives us idea what color is there on a first look, unless you know the Hue values of different colors. Still I prefer this method. It is

simple and better.

**Note**

While using this function, remember, interpolation flag should be nearest for better results.

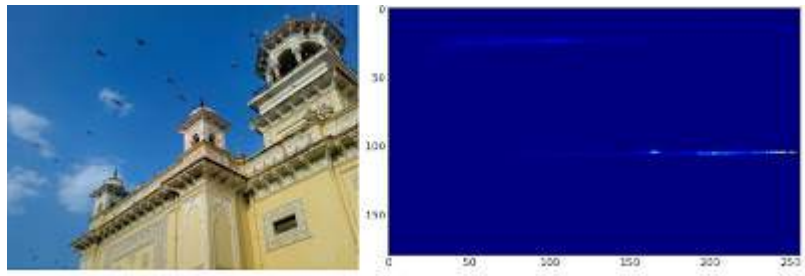
Consider code:

```
import numpy as np
import cv2 as cv
from matplotlib import pyplot as plt

img = cv.imread('home.jpg')
assert img is not None, "file could not be read, check with os.path.exists()"
hsv = cv.cvtColor(img,cv.COLOR_BGR2HSV)
hist = cv.calcHist( [hsv], [0, 1], None, [180, 256], [0, 180, 0, 256] )

plt.imshow(hist,interpolation = 'nearest')
plt.show()
```

Below is the input image and its color histogram plot. X axis shows S values and Y axis shows Hue.



**image**

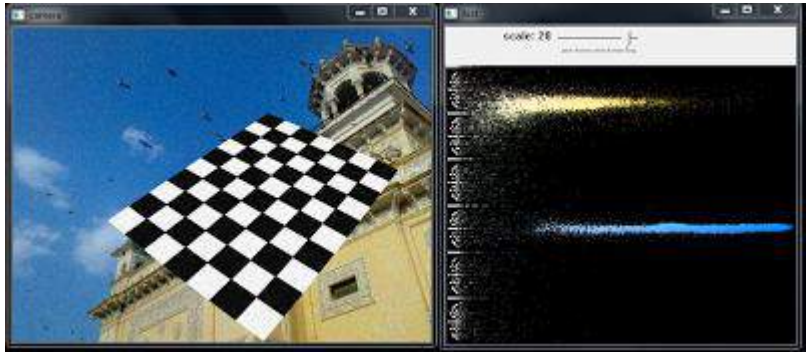
In histogram, you can see some high values near H = 100 and S = 200. It corresponds to blue of sky. Similarly another peak can be seen near H = 25 and S = 100. It corresponds to yellow of the palace. You can verify it with any image editing tools like GIMP.

## Method 3 : OpenCV sample style !!

There is a sample code for color-histogram in OpenCV-Python2 samples (samples/python/color_histogram.py). If you run the code, you can see the histogram shows the corresponding color also. Or simply it outputs a color coded histogram. Its result is very good (although you need to add extra bunch of lines).

In that code, the author created a color map in HSV. Then converted it into BGR. The resulting histogram image is multiplied with this color map. He also uses some preprocessing steps to remove small isolated pixels, resulting in a good histogram.

I leave it to the readers to run the code, analyze it and have your own hack arounds. Below is the output of that code for the same image as above:




**image**

You can clearly see in the histogram what colors are present, blue is there, yellow is there, and some white due to chessboard is there. Nice !!!

# Additional Resources

# Exercises

ENHANCED BY Google



# Histogram - 4 : Histogram Backprojection

## Goal

In this chapter, we will learn about histogram backprojection.

## Theory

It was proposed by **Michael J. Swain , Dana H. Ballard** in their paper **Indexing via color histograms**.

**What is it actually in simple words?** It is used for image segmentation or finding objects of interest in an image. In simple words, it creates an image of the same size (but single channel) as that of our input image, where each pixel corresponds to the probability of that pixel belonging to our object. In more simpler words, the output image will have our object of interest in more white compared to remaining part. Well, that is an intuitive explanation. (I can't make it more simpler). Histogram Backprojection is used with camshift algorithm etc.

**How do we do it ?** We create a histogram of an image containing our object of interest (in our case, the ground, leaving player and other things). The object should fill the image as far as possible for better results. And a color histogram is preferred over grayscale histogram, because color of the object is a better way to define the object than its grayscale intensity. We then "back-project" this histogram over our test image where we need to find the object, ie in other words, we calculate the probability of every pixel belonging to the ground and show it. The resulting output on proper thresholding gives us the ground alone.

## Algorithm in Numpy

1. First we need to calculate the color histogram of both the object we need to find (let it be 'M') and the image where we are going to search (let it be 'I').

```
import numpy as np
import cv2 as cvfrom matplotlib import pyplot as plt
```

```
#roi is the object or region of object we need to find
roi = cv.imread('rose_red.png')
assert roi is not None, "file could not be read, check with os.path.exists()"
hsv = cv.cvtColor(roi,cv.COLOR_BGR2HSV)

#target is the image we search in
target = cv.imread('rose.png')
assert target is not None, "file could not be read, check with os.path.exists()"
hsvt = cv.cvtColor(target,cv.COLOR_BGR2HSV)

# Find the histograms using calcHist. Can be done with np.histogram2d also
M = cv.calcHist([hsv],[0, 1], None, [180, 256], [0, 180, 0, 256] )
I = cv.calcHist([hsvt],[0, 1], None, [180, 256], [0, 180, 0, 256] )
```

2. Find the ratio $R = \frac{M}{I}$. Then backproject R, ie use R as palette and create a new image with every pixel as its corresponding probability of being target. ie B(x,y) = R[h(x,y),s(x,y)] where h is hue and s is saturation of the pixel at (x,y). After that apply the condition $B(x,y) = min[B(x,y), 1]$.

```
h,s,v = cv.split(hsvt)
B = R[h.ravel(),s.ravel()]
B = np.minimum(B,1)
B = B.reshape(hsvt.shape[:2])
```

3. Now apply a convolution with a circular disc, $B = D * B$, where D is the disc kernel.

```
disc = cv.getStructuringElement(cv.MORPH_ELLIPSE,(5,5))
cv.filter2D(B,-1,disc,B)
B = np.uint8(B)
cv.normalize(B,B,0,255,cv.NORM_MINMAX)
```

4. Now the location of maximum intensity gives us the location of object. If we are expecting a region in the image, thresholding for a suitable value gives a nice result.

```
ret,thresh = cv.threshold(B,50,255,0)
```

That's it !!

# Backprojection in OpenCV

OpenCV provides an inbuilt function **cv.calcBackProject()**. Its parameters are almost same as the **cv.calcHist()** function. One of its parameter is histogram which is histogram of the object and we have to find it. Also, the object histogram should be normalized before passing on to the backproject function. It returns the probability image. Then we convolve the image with a disc kernel and apply threshold. Below is my code and output :

```
import numpy as np
import cv2 as cv
```

```
roi = cv.imread('rose_red.png')
assert roi is not None, "file could not be read, check with os.path.exists()"
hsv = cv.cvtColor(roi,cv.COLOR_BGR2HSV)

target = cv.imread('rose.png')
assert target is not None, "file could not be read, check with os.path.exists()"
hsvt = cv.cvtColor(target,cv.COLOR_BGR2HSV)

# calculating object histogram
roihist = cv.calcHist([hsv],[0, 1], None, [180, 256], [0, 180, 0, 256] )

# normalize histogram and apply backprojection
cv.normalize(roihist,roihist,0,255,cv.NORM_MINMAX)
dst = cv.calcBackProject([hsvt],[0,1],roihist,[0,180,0,256],1)

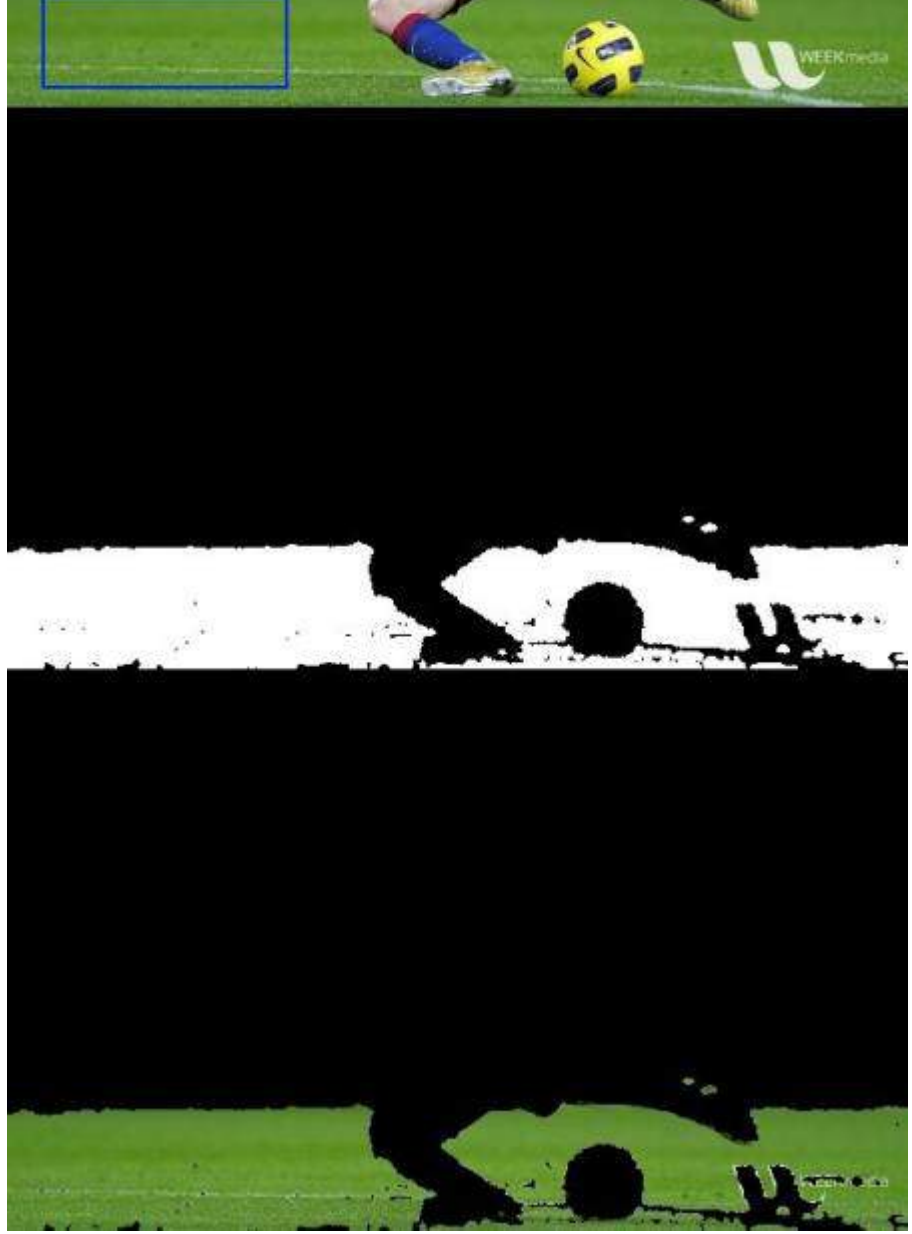
# Now convolute with circular disc
disc = cv.getStructuringElement(cv.MORPH_ELLIPSE,(5,5))
cv.filter2D(dst,-1,disc,dst)

# threshold and binary AND
ret,thresh = cv.threshold(dst,50,255,0)
thresh = cv.merge((thresh,thresh,thresh))
res = cv.bitwise_and(target,thresh)

res = np.vstack((target,thresh,res))
cv.imwrite('res.jpg',res)
```

Below is one example I worked with. I used the region inside blue rectangle as sample object and I wanted to extract the full ground.





**image**

# Additional Resources


1. "Indexing via color histograms", Swain, Michael J. , Third international conference on computer vision,1990.


# Exercises